TESPİT MÜHENDİSLİĞİ SERİSİ

PART 2

THREAD TELEMETRİ DESENKRONİZASYONU

EDR'ınızın Gördüğü ile Gerçekte Çalışan Arasındaki Fark

ÖZET

Önceki bölümde PEB manipülasyonunu ve bu manipülasyonun çekirdek ile kullanıcı modu telemetrisi arasında nasıl bir desenkronizasyon oluşturduğunu incelemiştik. Ayrıca bu yaklaşımın tek başına değil, farklı saldırı teknikleriyle birlikte zincirlenerek gerçek saldırı senaryolarında da kullanılabildiğinden bahsetmiştik.

Bu bölümde analizi bir adım daha ileri taşıyoruz. Süreç seviyesindeki manipülasyonların ötesine geçerek, thread düzeyinde oluşan telemetri farklılıklarını inceliyoruz. PEB sahteciliği, thread bağlam manipülasyonu, LOLBin zincirleme, Stack sahteciliği ve dolaylı syscall teknikleri bir araya geldiğinde saldırganın aynı süreç için birden fazla ve birbirleriyle tam olarak örtüşmeyen telemetri görüntüsü oluşturabildiğini göstereceğiz.

ODAK ALANI	TESPİT TÜRÜ
Thread / Süreç Telemetrisi	Çekirdek-Kullanıcı Desenkronizasyonu

Birkan KES

Tespit Mühendisi · Mart 2026 · AKTİF ARAŞTIRMA · v1.0

Devamı: "Loglara Ne Kadar Güvenebiliriz?" (Part 1, Şubat 2026)

▸ GİRİŞ

Part 1'in Bıraktığı Yerden

Bu serinin ilk bölümünde basit görünen bir soruyla başlamıştık: Loglara gerçekten ne kadar güvenebilirsiniz? POC testleri, PEB (Process Environment Block) manipülasyonunun çekirdeğin süreç oluşturma anında kaydettiği bilgiler ile kullanıcı modu araçlarının çalışma sırasında okuduğu bilgiler arasında belirgin bir tutarsızlık oluşturabildiğini gösterdi. `certutil.exe` senaryosu (Sysmon'da zararsız görünen bir LOLBin'in arka planda sessizce payload indirmesi) tek kaynağa dayalı tespit yaklaşımının tam olarak nasıl başarısız olduğunu ortaya koydu.

Ancak PEB manipülasyonu problemin yalnızca başlangıç noktasıdır. Asıl mesele, farklı telemetri katmanlarının aynı olayı her zaman aynı şekilde yansıtmamasıdır. Bu nedenle analizimizi süreç seviyesinden thread seviyesine taşıyoruz. Thread bağlamında ortaya çıkan telemetri desenkronizasyonu hem tespit edilmesi hem de farklı veri kaynaklarıyla ilişkilendirilmesi açısından çok daha karmaşık bir problem hâline getiriyor.

Temel soru dönüşüyor:

> ● Bir thread'in Win32StartAddress, RIP, call stack ve memory origin bilgileri birbirini doğrulamıyor ise
> **EDR hangisine güvenir?**

Bu makalede iki saldırı senaryosunu inceliyoruz. Senaryo 1, PEB sahteciliğini thread desenkronizasyonu ve bir LOLBin yürütme zinciri ile birleştirerek farklı telemetri katmanlarında tutarsız bir süreç görünümü oluşturur. Senaryo 2 ise kullanıcı modu kancalarına dayanan EDR tespitlerini atlatmak amacıyla yığın sahteciliğini dolaylı syscall tekniğiyle birlikte kullanır.

▸ ÇERÇEVE

6 Katmanlı Telemetri Modeli

Bir sürece ait güvenlik telemetrisi tek bir kaynaktan gelmez. İşletim sisteminin farklı katmanları, process lifecycle'ın farklı anlarında veri üretir. Bu nedenle aynı süreç hakkında birden fazla telemetri görünümü oluşur. Bu bir kusur değil, Windows'un mimari tercihidir. Sorun, savunucuların tüm katmanları eşdeğer kabul ettiğinde ya da saldırganların aralarındaki boşlukları istismar ettiğinde ortaya çıkar.

Layer	Captured Data	Timing	Manipulation Risk	Reliability
L0 Process Metadata	Sysmon Event 1 — kernel snapshot	Process creation time	Low — kernel recorded	★★★★★
L1 Thread Start Address	ETHREAD.Win32StartAddress	Thread creation time	Medium — stale after SetThreadContext	★★★
L2 Instruction Pointer	GetThreadContext → RIP (actual execution address)	Runtime	Low — live register state	★★★★
L3 Call Stack	RtlVirtualUnwind yığın çerçeve zinciri	Runtime	High — user-writable stack	★★
L4 Memory Origin	VirtualQuery.Type MEM_IMAGE — MEM_PRIVATE	Runtime	Medium — partial bypass possible	★★★
L5 Behavioral ETW	ETW Threat Intelligence Event 15 (SetThreadContext)	Event driven	Very Low — kernel event pipeline	★★★★★

Kritik içgörü: L1 ≠ L2 tutarsızlığı çoğu durumda gözlemlenebilir. Bir thread'in Recorded Start Address'i (L1) gerçek instruction pointer'ından (L2) farklı olduğunda, bu durum telemetri katmanları arasında ölçülebilir bir ayrışma üretir.

Stack spoofing (L3) ve indirect syscall (L4) bazı bireysel sinyalleri zayıflatabilir; ancak bu teknikler katmanlar arası tutarsızlığı tamamen ortadan kaldıramaz.

L5 — kernel ETW telemetry ise manipülasyonun kendisini gözlemler ve kernel-level yetki olmadan susturulması en zor katmanlardan biridir.

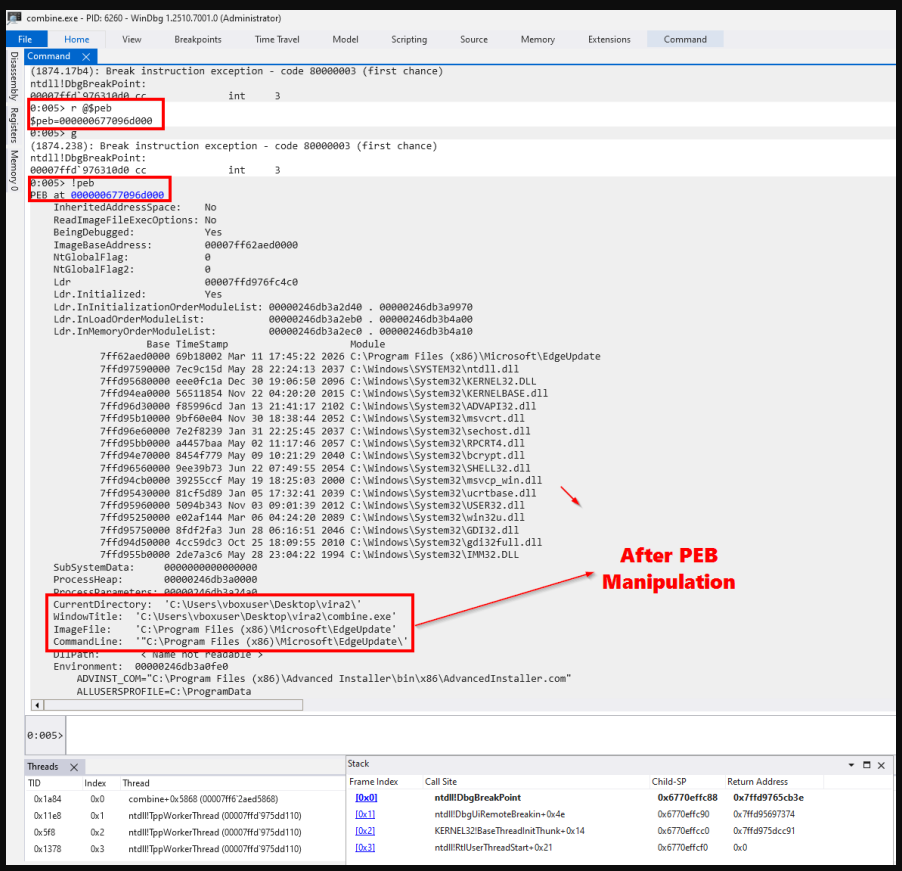
▸ SENARYO 1 · SALDIRI TEKNİĞİ

PEB Sahteciliği + Thread Desenkronizasyonu + LOLBin Zinciri

Senaryo 1, üç tekniğin zincirlenmesiyle oluşan telemetri ayrışmasını gösterir. Süreç PEB manipülasyonu ile sahte bir kimlik sunar; thread'ler suspended olarak oluşturulur ve yürütme başlamadan önce SetThreadContext ile yeniden yönlendirilir (L1/L2 desynchronization). Ardından gerçek yürütme meşru bir LOLBin üzerinden devam eder. Hangi telemetri katmanından bakıldığına bağlı olarak aynı süreç eş zamanlı olarak üç farklı kimlikle görünüyor.

3.1 PEB Kimlik Manipülasyonu

`combine.exe`, kendi PEB alanı içerisinde `RTL_USER_PROCESS_PARAMETERS` struct'ına doğrudan yazarak hem `ImagePathName` hem de `CommandLine` alanlarını `MicrosoftEdgeUpdate.exe`'ye işaret eden değerlerle değiştirdi. Part 1'de yalnızca CommandLine manipüle edilmişti; bu senaryoda `ImagePath` de manipüle edildi ve user-mode katmanında kimlik sahteciliği bu şekilde tamamlandı. Bunun sonucunda Task Manager, Process Explorer ve Procmon gibi araçlar PEB'i okuduklarında `combine.exe` yerine `MicrosoftEdgeUpdate.exe` çalışıyormuş gibi bir süreç kimliği görüntüler.

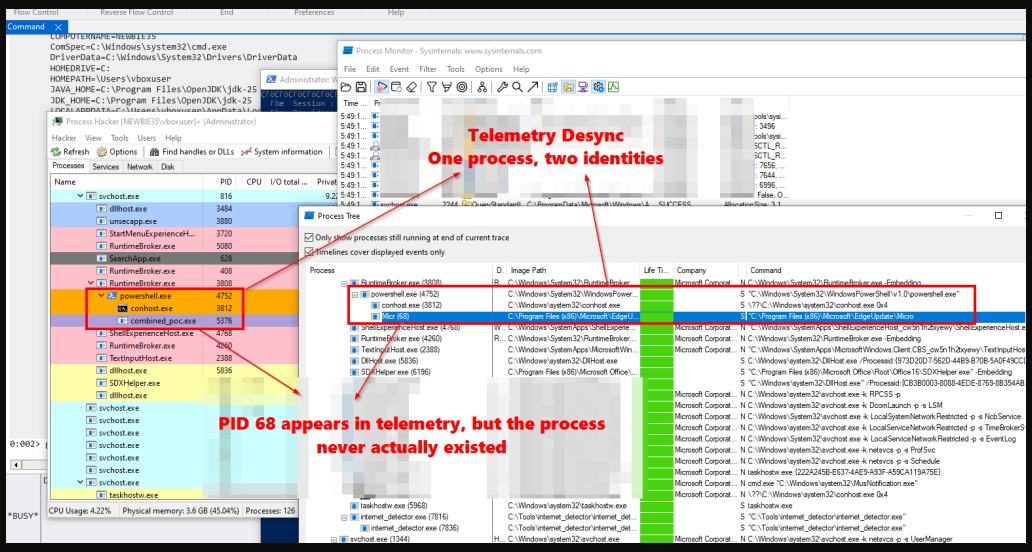


Şekil 1 — PEB manipülasyonu sonrası WinDbg çıktısı: hem ImageFile hem de CommandLine artık MicrosoftEdgeUpdate'e işaret ediyor. User-mode araçları manipüle edilmiş ImagePath'i gerçek process kimliği olarak okuyor. T-1'de yakalanan kernel kaydı değişmeden kaldı.

3.2 Procmon ve Process Hacker: Bir Süreç, İki Kimlik

Her iki aracın aynı süreç üzerinde eş zamanlı çalıştırılması mekanizmayı açıkça ortaya koyar. Procmon iki ayrı veri kaynağından bilgi toplar: kernel event akışı (PID 5376 = `combine.exe`, doğru) ve PEB tabanlı kullanıcı modu okumaları (ImagePath = `MicrosoftEdgeUpdate.exe`, manipüle edilmiş).

Procmon bu PEB verisini bağımsız bir kimlik olarak yorumladığında süreç listesinde ikinci bir giriş oluşturur. Ortaya çıkan sonuç bir "ghost process" durumudur: PID 68 gerçekte hiç var olmamış (hiçbir log bulunmamakta), yalnızca yanlış ilişkilendirilmiş PEB okumalarından dolayı üretilmiş bir süreçtir.



Şekil 2 — Procmon (sağda) ve Process Hacker (solda): aynı process, iki kimlik. PID 68 Procmon telemetrisinde görünüyor ancak hiçbir kernel-level process listesinde karşılık bulunamıyor.

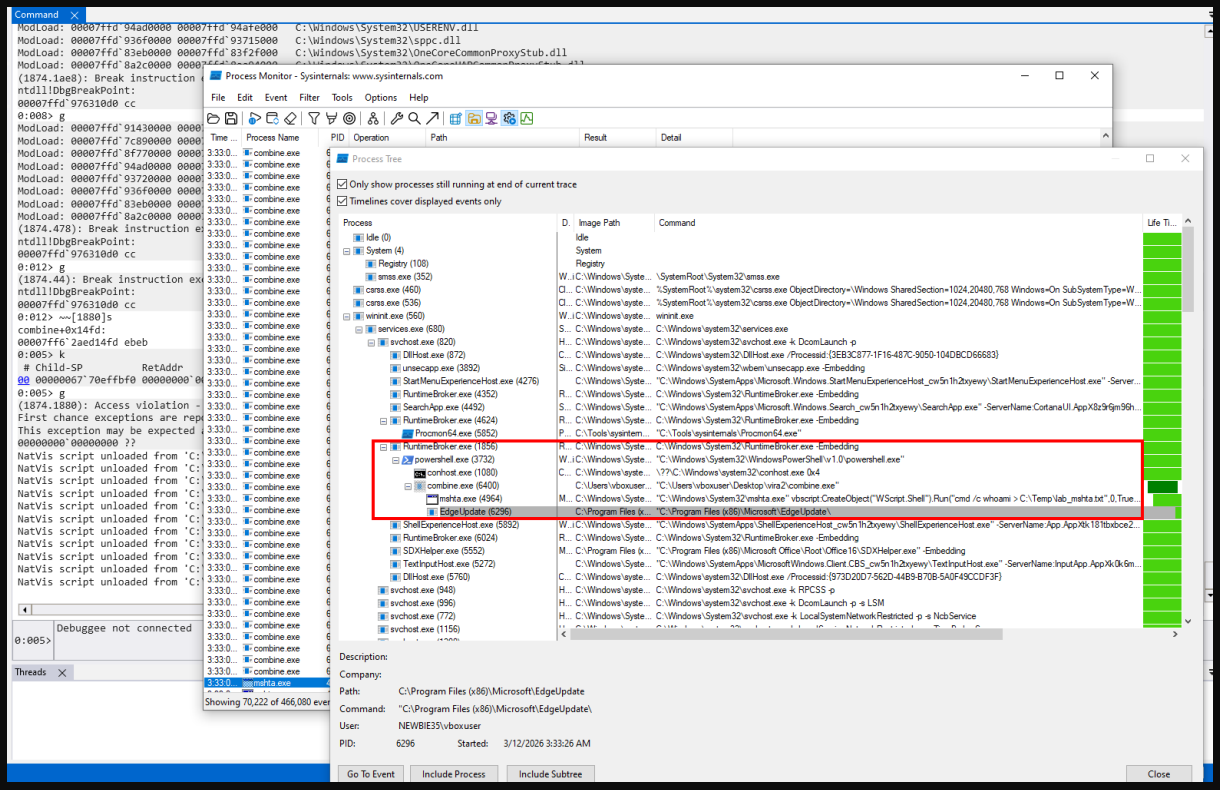
> ⚠ Temel Bulgu: Procmon, parent process zincirlerini PEB'den okuyor; bu durum process tree analizi için tek başına güvenilir değildir. Sysmon Event ID 1 her zaman Process Creation Time kaydını yansıtır.

3.3 Üç Telemetri Kaynağı, Üç Farklı Hikâye

`mshta.exe`, yürütme zincirinin son aşamasında başlatıldığında üç farklı telemetri görünümü aynı anda gözlemlendi. Her araç kendi telemetri katmanından gördüğünü doğru raporladı.

Ancak bu katmanlar aynı olay için birbiriyle uyumsuz üç farklı yürütme anlatısı üretti:

Telemetry Source	What Does It Show?	Ground Truth
Telemetry 1 (Kernel / Sysmon)	combine.exe → mshta.exe execution chain	✓ Correct
Telemetry 2 (PEB View)	EdgeUpdate (PID 6296) → mshta.exe (PID 6296 hiç var olmadı)	× Ghost
Telemetry 3 (Mixed Telemetry)	combine.exe → mshta.exe + child process EdgeUpdate (EdgeUpdate için kernel kaydı yok)	~ Misleading



Şekil 3 — Aynı yürütme zinciri farklı telemetri katmanlarında üç farklı süreç gösteriyor.

Bu tutarsızlık yalnızca üç telemetri görünümü birlikte korele edildiğinde ortaya çıkar. Telemetri ayrı ayrı değerlendirildiğinde ise her sinyal bağımsız olarak zararsız görünebilir.

▸ SENARYO 2 · SALDIRI TEKNİĞİ

Stack Spoofing + Indirect Syscall

Senaryo 2, süreç meta verisinin altındaki tespit katmanlarını hedef alıyor. Bir sürecin nasıl göründüğünü değiştirmek yerine API çağrılarının bir EDR tarafından nasıl gözlemlendiğini değiştiriyor. Indirect syscall tekniği hooklanmış ntdll fonksiyonlarını atlayarak çağrının doğrudan kernel'e gitmesini sağlıyor. Stack spoofing ise thread'in call stack'ini meşru bir çağrı zinciri gibi gösterecek şekilde manipüle ediyor. Birlikte kullanıldıklarında L3 (call stack) ve L4 (memory origin) telemetrilerini kısmen atlatabiliyor; ancak L5'te, yani behavioral ETW telemetry katmanında gözlemlenebilir bir iz bırakır.

4.1 EDR Hooks Neden Hedef Alınıyor?

Günümüzde tespit platformları kernel telemetrisi, `ETW` ve bellek incelemesini birlikte kullanmaktadır. Ancak birçok davranışsal sinyal hâlâ user-mode yürütme yollarından kaynaklanır. Bu nedenle `indirect syscall` gibi teknikler ofansif araştırmalarda hâlâ önemini korumaktadır. Söz konusu yaklaşımda ntdll.dll içindeki NTAPI fonksiyonlarının entry point'lerine küçük bir `JMP patch` kodu yerleştirilir.
Bir process `NtQueryVirtualMemory` gibi bir fonksiyonu çağırdığında öncelikle bu hook'a yönlendirilir; EDR çağrının context'ini inceleyerek devam edip etmeyeceğine karar verir.

Bu mekanizmayı atlatmak için hook'lu fonksiyonu tamamen atlayıp syscall'ı doğrudan kernel'e iletmek gerekir. Bunun için iki bileşen gereklidir:

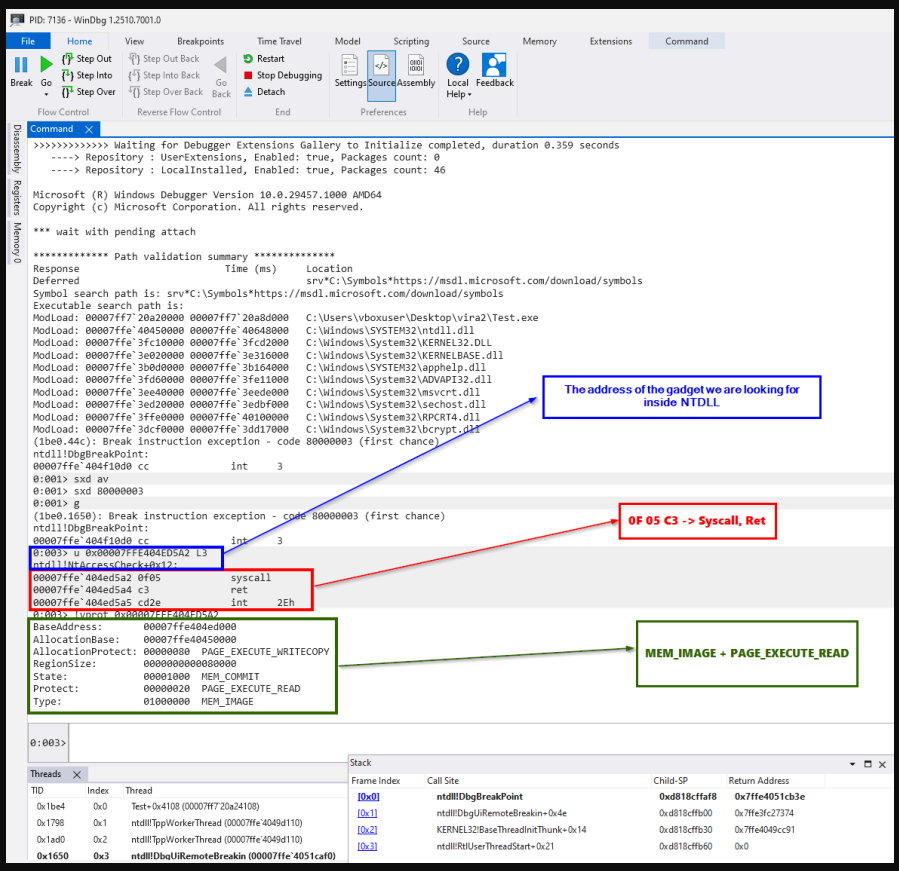
- Hedef fonksiyonun System Service Number (SSN) değeri
- syscall talimatına ulaşmayı sağlayan geçerli bir stub veya ret gadget

4.2 FreshyCalls: SSN Çözümleme + Gadget Tespiti

PoC, ntdll.dll export tablosunu parse ederek tüm Nt* fonksiyonlarını virtual address'e göre sıralıyor ve SSN'leri sıra pozisyonuyla eşliyor. Test ortamında `NtQueryVirtualMemory` için SSN değeri saptandı. `0F 05 C3` (syscall; ret) gadget'ı `ntdll!NtAccessCheck+0x12` adresinde tespit edildi.

```
mov r10, rcx ; syscall çağrı kuralı
mov eax, 27h ; SSN = 39 → NtQueryVirtualMemory (build-specific)
mov r11, ntdll+0x12 ; syscall;ret gadget adresi (EDR inline hook atlanıyor)
jmp r11 ; execution ntdll MEM_IMAGE bölgesine aktarılıyor
```

Gadget adresinin kritik özelliği `MEM_IMAGE + PAGE_EXECUTE_READ` olarak resolve edilmesidir. ntdll'nin Microsoft imzalı, mapped belleği. Syscall çalıştığı an RIP bu bölgeye işaret ediyor. EDR'ın memory-type kontrolü meşru, non-writable bir image page görüyor; normalde alarm tetikleyen `MEM_PRIVATE` veya RWX bölgeyi değil. Sonuç olarak L4 kısmen atlatılıyor.



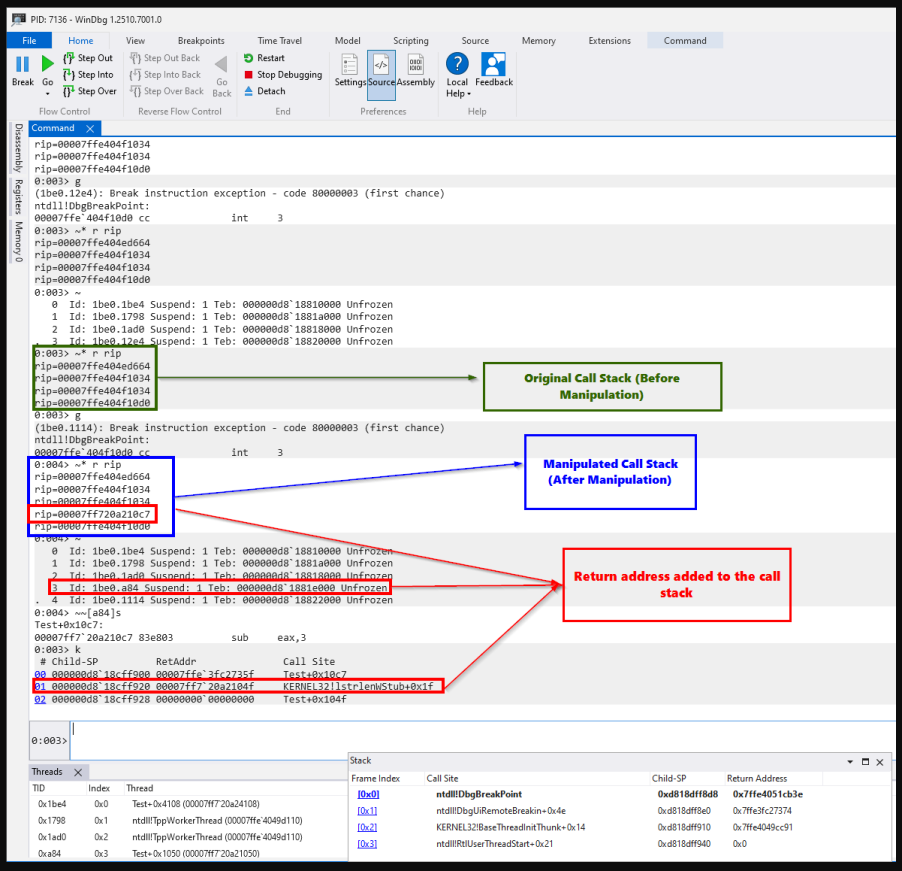
Şekil 4 — WinDbg: ntdll!NtAccessCheck+0x12 adresinde syscall;ret gadget'ı. Bellek bölgesi ayrıntılarında MEM_IMAGE + PAGE_EXECUTE_READ görünüyor; EDR'ın köken perspektifinden bakıldığında bu tamamen meşru (NTDLL) bir kod.

Memory Type	Description	EDR Response
MEM_PRIVATE + RWX	Application-allocated, writable + executable — classic shellcode staging	⚡ ALARM
MEM_PRIVATE + RX	(Just-In-Time Compilation) JIT-compiled code; suspicious if no known JIT engine present	▯ INSPECT
MEM_IMAGE + RX	Mapped from signed PE — expected execution location	✓ ALLOW

4.3 Yığın Sahteciliği: Derinlik Anomalisini Giderme

Indirect syscall'lar memory origin sorununu çözüyor ancak yeni bir sorun yaratıyor: thread'in call stack'i sığ kalıyor. Meşru bir API çağrısı kernel32.dll'den geriye doğru uzanan, RtlUserThreadStart'a kadar izlenebilen bir stack üretiyor. Doğrudan bir gadget atlaması ise çok az frame bırakıyor; stack depth tespit kuralları bunu yakalıyor.

Stack spoofing, thread'in stack'ine sahte return address'ler yazarak bunu gideriyor (SetThreadContext ile RSP kaydırılıyor). PoC, BaseThreadInitThunk ve RtlUserThreadStart'ı ek frame olarak ekledi. Stack depth kontrolünü artık geçiyor; stack'te üç frame mevcut, her ikisi de kernel32 ve ntdll'ye işaret ediyor. Ancak kritik `kernel32!NtQueryVirtualMemory` frame'i eksik — image_rop anomalisi olarak tespit edilebilir.

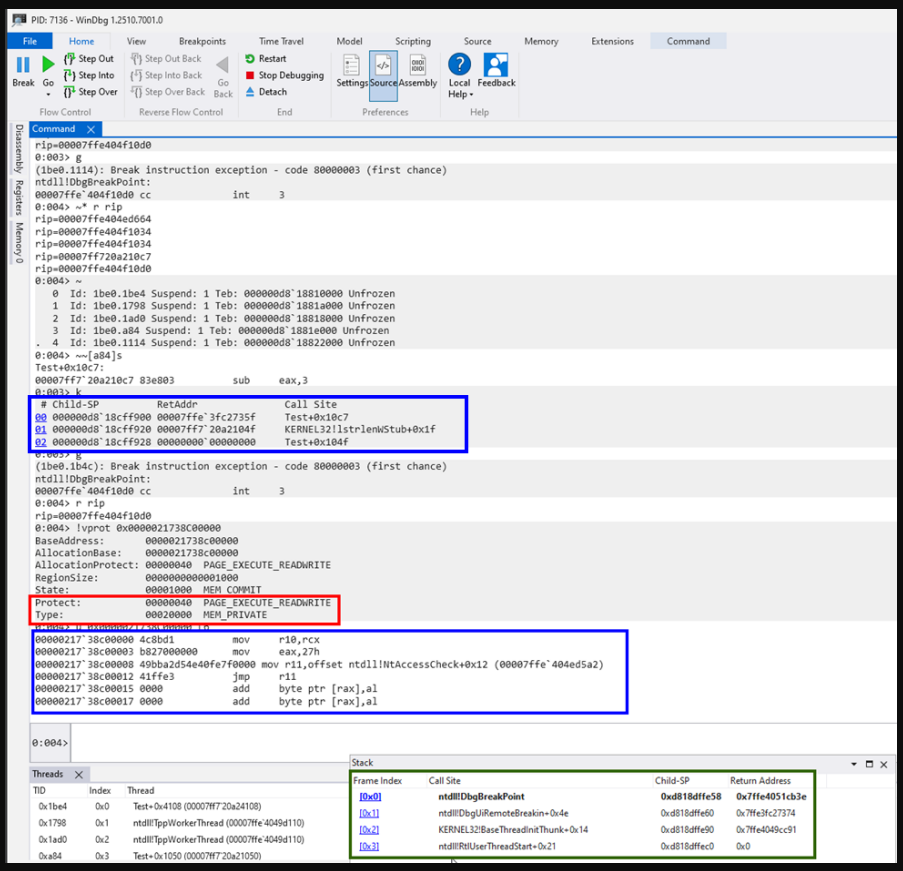


Şekil 5 — Sahtecilik öncesi ve sonrası çağrı yığını (WinDbg). 2 çerçeve derinlik anomalisi görsel olarak gideriliyor, ancak kernel32!NtQueryVirtualMemory çağrı zincirinde yok — Eclipse CALL-öncesi-dönüş denetimi bunu yakalıyor.

4.4 Stub: MEM_PRIVATE Görünür Kalmaya Devam Ediyor

Indirect syscall'ı yöneten stub, RWX izinlerine sahip `MEM_PRIVATE` bellekte yaşıyor. İçeriği: `mov r10, rcx` → `mov eax, 27h` → `jmp ntdll_gadget` — doğrudan taranabilir. Process belleğini gezerek executable private page'leri inceleyen bir EDR bu stub'ı tespit edecektir. Bu nedenle teknik tam bir bypass değil, kısmi bir atlatmadır.

Ancak pratikte `MEM_PRIVATE + RWX` tespiti EDR'lar için doğrudan alarm üretmekten çok ikincil analizi tetikleyen bir sinyal olarak çalışır. False positive oranı yüksek olduğundan çoğu vendor bu kontrolü bağımsız bir engelleme kuralı olarak değil, diğer sinyallerle korelasyon kurulduğunda devreye giren bir gösterge olarak kullanır.



Şekil 6 — WinDbg'den okunan stub içeriği. Bellek bölgesi ayrıntılarında MEM_PRIVATE + PAGE_EXECUTE_READWRITE görünüyor; bir EDR tarayıcısı bu bölgeyi doğrudan bulup analiz edebilir.

× Kısmi Atlatma: Bu teknik kombinasyonu L4'ü (Memory Origin) tam olarak geçemiyor. Indirect syscall execution'ı MEM_IMAGE bölgesine taşısa da MEM_PRIVATE stub bellekte görünür kalmaya devam ediyor. L3'te (Call Stack) eksik kernel32!NtQueryVirtualMemory frame'i image_rop anomalisi olarak tespit ediliyor. L5'te (Behavioral ETW) ise SetThreadContext her çağrıldığında ETW TI Event 15 tetikleniyor ve sinyal oluşturuyor.

▸ ANALİZ

Senaryo Karşılaştırması: Katman Katman

Aşağıdaki tablo her telemetri katmanını her iki senaryo karşısında eşliyor; hangi katmanların atlatıldığını, hangilerinin alarm ürettiğini ve hangilerinin manipülasyona yapısal olarak bağışık kaldığını gösteriyor.

Layer	Senaryo 1 — PEB + Thread Desync	Senaryo 2 — Stack Spoofing + Indirect Syscall	Bypassed?
L0 — Process Metadata (Sysmon Event 1)	UNCHANGED — kernel record immutable	UNCHANGED — kernel record fixed	NO
L1 — Thread Start Address (Win32StartAddress)	OUTDATED — set at thread creation, not updated post-redirect	OUTDATED — same mechanism	PARTIAL
L2 — Instruction Pointer (GetThreadContext.Rip)	TRUTH — current execution address	TRUTH — current execution address	NO
L3 — Call Stack (RtlVirtualUnwind)	SHORT STACK anomaly — only 2 frames	FAKE STACK — looks normal but chain incomplete	PARTIAL
L4 — Memory Origin (VirtualQuery.Type)	MEM_IMAGE — normal	Gadget = MEM_IMAGE ✓ Stub = MEM_PRIVATE ✕	PARTIAL
L5 — Behavioral ETW (TI Event 15)	⚡ ALARM	⚡ ALARM	NO
PEB Identity — User-mode tools	SPOOFED — EdgeUpdate	—	YES (user-mode only)

Her iki senaryoda pattern tutarlı: hiçbir teknik tek başına 6 katmanın tamamını atlatamıyor. L0 (Process Metadata) ve L2 (Instruction Pointer) yapısal olarak immune; kernel kayıtları user-mode manipülasyonla değiştirilemiyor. L5 (Behavioral ETW) her SetThreadContext çağrısında teknikten bağımsız tetikleniyor. L3 (Call Stack) ve L4 (Memory Origin) kısmen atlatılabiliyor; ancak her ikisinde de iyi yapılandırılmış bir detection stack'in tanımlayabileceği artık sinyaller kalıyor.

> ⚠ Ancak Microsoft-Windows-Threat-Intelligence provider'ı varsayılan olarak kapalı gelir; aktif edilmemiş veya SIEM tarafında toplanmamış ortamlarda bu sinyal savunucuya ulaşmaz.

▸ SAVUNMA REHBERİ · MAVİ TIM

Asgari Tespit Seti

Aşağıdaki dört sinyal, bu laboratuvarda gösterilen her tekniği tespit etmek için bir arada yeterlidir. Hiçbiri özel kernel driver gerektirmiyor. Tamamı Sysmon, ETW ve cross-source korelasyon yapabilen bir SIEM ile uygulanabilir.

#	Signal	Source	What It Catches	Priority
1	ETW TI Event 15 (SetThreadContext)	ETW ThreatIntelligence provider	Every SetThreadContext call — both scenarios trigger here unconditionally	CRITICAL
2	L1/L2 Cross-Check (Win32StartAddress ≠ RIP)	NtQueryInformationThread + GetThreadContext	Thread desync — start address diverges from actual instruction pointer	HIGH
3	Sysmon Event 1 (Process Creation)	Sysmon kernel callback — immutable	Actual Image, CommandLine, ParentImage — immune to all PEB modifications	HIGH
4	Abnormal stack depth + missing expected frames in CALL chain	Call stack analysis	Abnormal thread origin — detects spoofed frames even when depth appears normal	MEDIUM

Savunma Mimarisi: Beş Soru

Bu tekniklere karşı etkili savunma kural listesinden çok, ilgilenilen her süreç için beş soruyu sırayla sormayı gerektiriyor:

S1 **Bu surec gercekte nasil basladi?**

```
PsCreateProcessNotifyRoutineEx
Immutable kayit, olusturulduktan sonra degistirilemez.
```

↓

S2 **Bize o anda ne bildirildi?**

```
Sysmon Event 1
CommandLine ve Image alanlarini kernel callback verisiyle karsilastir -> Mismatch = ALARM
```

↓

S3 **Bu process calisirken ne yapti?**

```
Sysmon Event 10  |  API call telemetrisi  |  Injection sinyalleri
Declared identity'den sapan davranis guclu bir gosterge.
```

↓

S4 **Bu davranis baglamsal acidan mantikli mi?**

```
Parent-child chain analizi  |  LOLBin call'lari
SetThreadContext event'lari ile network aktivitesi arasinda time correlation.
```

↓

S5 **Tum kaynaklar uyusuyor mu?**

```
Kernel <-> user-mode  |  Log <-> memory  |  Event <-> davranis
Layer'lar arasindaki discrepancy sinyalin kendisidir.
```

▸ SONUÇ

Logs Are Not Wrong. They're Incomplete.

Part 1 ve Part 2'de belgelenen teknikler ortak bir tasarım ilkesini paylaşıyor: log'ları silmiyorlar, EDR process'lerini çöktürmüyorlar, kernel exploit'e ihtiyaç duymuyorlar. Yaptıkları şey, tek bir telemetri layer'ını gözlemleyen bir savunucunun internally tutarlı, teknik olarak doğru ve temelden yanıltıcı bir şey göreceği gerçeğini istismar etmektir.

PEB, user-mode araçlarının güvendiği değiştirilebilir metadata barındırıyor. Thread'in Win32StartAddress'i bir kez yazılıyor ve bir daha güncellenmiyor. Call stack user-mode tarafından yazılabilir bellek. Bunların hiçbiri security vulnerability değildir; kullanılan design decision'lar ve saldırganların bulduğu gibi tam yerinde durmaya devam ediyorlar. Değişen şey bu kararların exploit edilme karmaşıklığı ve bununla eşleşmesi gereken detection kalitesidir.

> *"Saldırganın amacı saklanmak değil. Algıyı bozmak — kötü niyetli davranışı savunucunun zaten güvendiği bir şey gibi göstermek."*

Temel Çıkarımlar

✓ **L1 (Thread Start Address) ≠ L2 (Instruction Pointer)**
Tek başına L2 (instruction pointer) alarm üretmek için güvenilir bir gösterge değildir; runtime manipülasyonlara açıktır ve false positive üretebilir. Ancak L1 (Win32StartAddress) ile L2 arasındaki sapma, tüm thread manipülasyon senaryolarında tespit edilebilir. Bu tespit için özel araçlar gerekmez; mevcut kernel ve user-mode telemetry verilerini kullanarak cross-layer korelasyon yeterlidir.

✓ **L5 (Behavioral ETW — TI Event 15) en yüksek güvenilirlikli kernel sinyali**
PoC kapsamında test edilen her SetThreadContext çağrısı bu event'ı üretti. Microsoft-Windows-Threat-Intelligence provider doğru yapılandırılmışsa thread hijacking sessizce gerçekleştirilemez. Bu provider varsayılan olarak kapalı geliyor; EDR ve SIEM'lerinizde bu provider'ı aktif etmeyi ve Event 15'i bir detection sinyali olarak toplamayı unutmayın.

✓ **Stack spoofing (L3 — Call Stack) depth'i düzeltiyor ama CALL chain integrity'yi bozuyor**
CALL-öncesi-dönüş kontrolü uygulanırsa kaç frame olursa olsun sahte frame'leri yakalıyor. Stack depth tek başına yeterli bir detection kriteri değildir.

✓ **Korelasyon zorunludur, opsiyonel değil**
Hiçbir araç tek başına tam resmi göremedi. Saldırı ancak kernel ve user-mode verisi birbirine karşı tutulduğunda görünür hale geliyor. SIEM'iniz bunu otomatik yapamıyorsa, kapatılacak ilk gap budur.

▸ KAYNAKLAR VE REFERANSLAR

- Birkan KES — "Loglara Ne Kadar Güvenebiliriz? Çekirdek-Kullanıcı Telemetri Desenkronizasyonu" (PART 1, Şubat 2026)
- KlezVirus — "Eclipse: CALL/Dönüş Bütünlüğü ile Yığın Sahteciliğini Tespit Etme" (2024)
- Alice Climent (RedOps) — "Dolaylı Syscall ve Yığın Sahteciliğiyle EDR Atlatma" (2024)
- LetsMakeMalware — "FreshyCalls: Syscall Stub Ayrıştırma ve SSN Çözümleme"
- Elastic Security Labs — "Kernel ETW is the Best ETW" (2024)
- Microsoft — PsSetCreateProcessNotifyRoutineEx2 / PS_CREATE_NOTIFY_INFO
- Microsoft — Windows Internals, 7. Baskı: ETHREAD.Win32StartAddress davranışı
- Trail of Bits — "Güvenlik Araştırması ve Adli Bilişim için ETW Dahilileri" (2023)
- Velociraptor — "ETW Bölüm 2: Süreç Üst Sahteciliği" (2021)